

Project To-Do List

A Beginner-Friendly Streamlit Project

A mentoring guide: what the project is, the ideas it teaches, how the code works, and exercises to extend it yourself.

1. What You're Building

A simple to-do list web app: add tasks, mark them done, delete them, and see your progress -- all running in the browser, built entirely in Python using a framework called **Streamlit**. No HTML, CSS, or JavaScript required.

The tasks are saved to a small JSON file on disk, so if you close the browser tab and come back later, your list is still there.

2. What Is Streamlit, and Why Is It Different?

Most web apps you've used follow a request/response model: the browser asks the server for something, the server sends back an answer, and nothing changes until the next request. Streamlit works differently, and this is the single most important idea to understand before reading the code:

Every time you interact with a Streamlit app -- click a button, type in a text box, tick a checkbox -- Streamlit re-runs your *entire* Python script from top to bottom. The whole file, every single time.

This sounds wasteful, but it's what makes Streamlit so simple to write: you just describe what the page should look like **right now**, given the current data, and Streamlit figures out how to redraw it. There's no separate step where you manually update parts of the page -- you just write normal top-to-bottom Python.

The catch: if the whole script re-runs constantly, how does the app remember anything (like the list of tasks) between reruns? That's what `st.session_state` is for -- covered in section 4.

3. Project Structure

```
meharu-todolist/
  app.py           <- the whole app lives in this one file
  requirements.txt  <- Python packages needed (streamlit, reportlab)
  todos.json       <- created automatically -- your saved tasks
  .venv/           <- virtual environment (isolated Python packages)
```

Keeping everything in one file is a deliberate choice for a first project -- once you're comfortable with how it works, splitting it into multiple files (e.g. a separate `storage.py`) is a natural next step (see the exercises).

4. Code Walkthrough

4.1 Loading and saving data

Two small functions handle everything to do with the file on disk:

```
def load_todos():
    if not os.path.exists(DATA_FILE):
        return []
    with open(DATA_FILE) as f:
        return json.load(f)

def save_todos(todos):
    with open(DATA_FILE, 'w') as f:
        json.dump(todos, f, indent=2)
```

Each task is just a Python dictionary: `{'title': ..., 'priority': ..., 'done': False, 'created': ...}`. The whole list of tasks is a list of these dictionaries, which JSON can store directly.

4.2 Remembering things across reruns: `st.session_state`

Since the script re-runs on every click, a plain Python variable like `todos = []` would reset to empty every single time. `st.session_state` is a dictionary-like object that Streamlit keeps alive for you across reruns, for as long as the browser tab stays open:

```
if 'todos' not in st.session_state:
    st.session_state.todos = load_todos() # only runs on the very first load
```

After this line, the rest of the script can read and modify `st.session_state.todos` freely, and it'll still be there on the next rerun.

4.3 The add-task form

A `st.form(...)` groups the text input, the priority dropdown, and the submit button together, so the app only reacts once (when 'Add task' is clicked) rather than on every keystroke:

```
with st.form('add_task_form', clear_on_submit=True):
    title = st.text_input('New task')
    priority = st.selectbox('Priority', PRIORITIES)
    submitted = st.form_submit_button('Add task')

    if submitted and title.strip():
        st.session_state.todos.append({...})
        save_todos(st.session_state.todos)
```

4.4 Displaying and editing the list

The app loops over every task and draws a row of widgets for it: a checkbox, the title, the priority, and a delete button. Each widget needs a unique `key=` (built from the task's index, e.g. `f'done_{i}'`) so Streamlit can tell rows apart across reruns.

When something changes (a checkbox is ticked, a task is deleted), the code updates `st.session_state.todos`, saves it to disk, and calls `st.rerun()` to immediately redraw the page with the new state -- rather than waiting for the *next* natural rerun.

5. Running the App

```
cd meharu-todolist
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/streamlit run app.py
```

Streamlit will print a local URL (usually `http://localhost:8501`) -- open it in a browser to use the app.

6. Exercises -- Extend It Yourself

Try these roughly in order -- each one builds on ideas already in the code:

- 1 Add a "due date" field to each task (`st.date_input`) and show overdue tasks in red.
- 2 Add an "Edit" button that lets you change a task's title in place.
- 3 Sort tasks by priority (High first) instead of the order they were added.
- 4 Add a search box (`st.text_input`) that filters the visible tasks by title.

- 5 Show a simple bar chart (`st.bar_chart`) of how many tasks are High/Medium/Low priority.
- 6 Split the file: move `load_todos/save_todos` into a separate `storage.py` and import it.
- 7 Swap the JSON file for a small SQLite database (Python's built-in `sqlite3` module) -- same idea, more scalable storage.
- 8 Add multiple named lists (e.g. "Work", "Personal") using `st.sidebar` and a `selectbox`.

7. Key Takeaways

- Streamlit re-runs the whole script on every interaction -- design around that, don't fight it.
- `st.session_state` is how an app remembers things across those reruns.
- Widgets need unique keys when you create several of the same kind in a loop.
- Persisting data is as simple as reading/writing a file -- you don't need a database to start.